

Extended Runs 1

Philip Shea

2024-09-23

New Runs

The following Runs (and by “run” is meant a particle system initialized at an energy and propagated from there; when temperature changes are included, each temperature change would be a new run) were extended by 15,000 steps.

```
[1] "0016" "0021" "0023" "0030" "0031" "0036" "0038" "0037" "0051" "0047" "0052"  
[12] "0056" "0059" "0053" "0066" "0067" "0082" "0073" "0074" "0079" "0083" "0092"
```

Here, for example, is the extended 31 series.

```
x31 = combineruns( "H:/Data/DSP/MD/Data/1980", "S2\\0031\\")  
plotMD2( tail( x31, -2000))
```

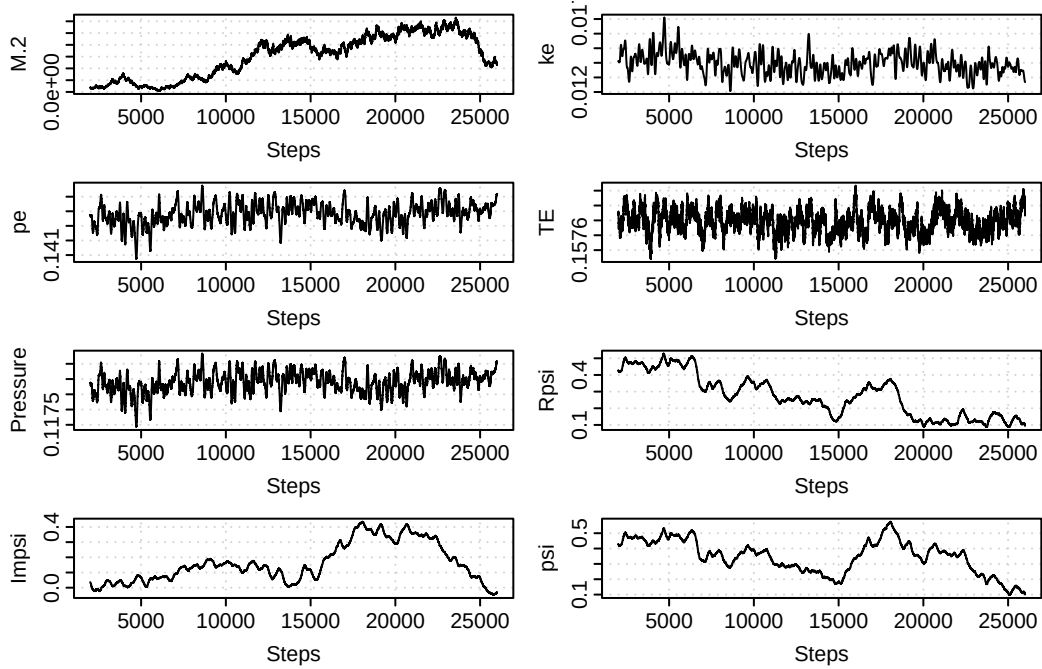


Figure 1: Plots of the basic data in run S2.0031 after skipping the first 2,000 points.

The fit to the temperature works, but is suspect.

```
navg <-500
skip <- 2000
(t <- procMD2DF( x31, skip=skip, rsblk=500))
```

sdf: Total Size: 24000, Step: 2048, Number of FFTs: 10, window Power: 4096

total.steps	steps.avg	TE.init	xMomentum.mean.rs
2.600000e+04	1.600000e+04	1.585944e-01	8.820070e-11
xMomentum.sd.rs	yMomentum.mean.rs	yMomentum.sd.rs	M.2.mean.rs
4.861720e-11	1.886244e-11	2.042276e-11	1.088329e-20
M.2.sd.rs	ke.mean.rs	ke.sd.rs	pe.mean.rs
7.450399e-21	1.401502e-02	4.659884e-04	1.437791e-01
pe.sd.rs	TE.mean.rs	TE.sd.rs	Pressure.mean.rs
4.521987e-04	1.577941e-01	3.704942e-05	1.188769e-01
Pressure.sd.rs	Rpsi.mean.rs	Rpsi.sd.rs	Impsi.mean.rs
2.122264e-04	3.365456e-01	9.912047e-02	1.162985e-01
Impsi.sd.rs	psiabs.mean.rs	psiabs.sd.rs	xMomentum.min.raw
9.056868e-02	3.683442e-01	9.574136e-02	-9.571455e-12

xMomentum.max.raw	yMomentum.min.raw	yMomentum.max.raw	M.2.min.raw
1.561237e-10	-3.100786e-11	7.483036e-11	3.916244e-22
M.2.max.raw	ke.min.raw	ke.max.raw	pe.min.raw
2.437940e-20	1.206789e-02	1.709850e-02	1.407282e-01
pe.max.raw	TE.min.raw	TE.max.raw	Pressure.min.raw
1.457594e-01	1.575401e-01	1.580339e-01	1.174270e-01
Pressure.max.raw	Rpsi.min.raw	Rpsi.max.raw	Impsi.min.raw
1.198350e-01	1.203260e-01	5.280718e-01	-2.331094e-02
Impsi.max.raw	psiabs.min.raw	psiabs.max.raw	nr
4.297305e-01	1.697818e-01	5.677285e-01	3.200000e+01
Estimate	Std.Error	r.squared	Fval
-2.126823e-06	7.418937e-06	2.731934e-03	8.218252e-02
Fpvalue	df.dendf	rtv	psiavg
7.763318e-01	3.000000e+01	1.105510e-03	3.560734e-01
specwidth			
2.636719e-02			

```
rx31ke <- rsmavg( tail( x31$ke, -skip), navg)
rx31 <- data.frame( ke=rx31ke,
                    step=seq( from=navg/2, by=navg, along.with=rx31ke))
lmrx31 <- lm(ke ~ step, data = rx31) # fit across entire data set
(slmrx31 <- summary( lmrx31))
```

Call:

```
lm(formula = ke ~ step, data = rx31)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-8.602e-04	-3.560e-04	2.106e-05	3.275e-04	8.039e-04

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	1.431e-02	1.178e-04	121.469	< 2e-16 ***
step	-2.848e-08	8.501e-09	-3.351	0.00162 **

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.000408 on 46 degrees of freedom

Multiple R-squared: 0.1962, Adjusted R-squared: 0.1787

F-statistic: 11.23 on 1 and 46 DF, p-value: 0.001618

```
#lenrx31 <- nrow( rx31)
#third <- floor(lenrx31 / 3) # we'll exclude the least number of points
trx31 <- tail( rx31, t['nr']) # procMD2DF fit
sstrx31 <- stat.summary( trx31$ke)
lmtrx31 <- lm( ke ~ step, data=trx31) # fit to tail
summary( lmtrx31)
```

Call:

```
lm(formula = ke ~ step, data = trx31)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-9.510e-04	-2.941e-04	-4.804e-05	3.068e-04	6.814e-04

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	1.390e-02	2.471e-04	56.239	<2e-16 ***
step	-4.254e-09	1.484e-08	-0.287	0.776

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.0003875 on 30 degrees of freedom

Multiple R-squared: 0.002732, Adjusted R-squared: -0.03051

F-statistic: 0.08218 on 1 and 30 DF, p-value: 0.7763

```
ftlmx31 <- lm( ke ~ step, data=head( rx31, -t['nr'])) # Fit to head of data.
plot( tail( x31$ke, -skip), type='l', panel.first=grid()) # plot raw data
points(ke ~ step, data=rx31, col='darkred') # Plot all resample points
abline( coef=coef( lmr31)) # Fit to all resample points
points(ke ~ step, data=trx31, col='blue') # Plot tail resample points
abline( coef=coef( lmtrx31), col='blue') # Fit to the tail resample points
abline( h=sstrx31["mean"], col='darkgreen')
abline( h=sstrx31["min"], col="darkgreen", lty=2)
abline( h=sstrx31["max"], col="darkgreen", lty=2)
abline( coef=coef( ftlmx31), col='darkred')
```

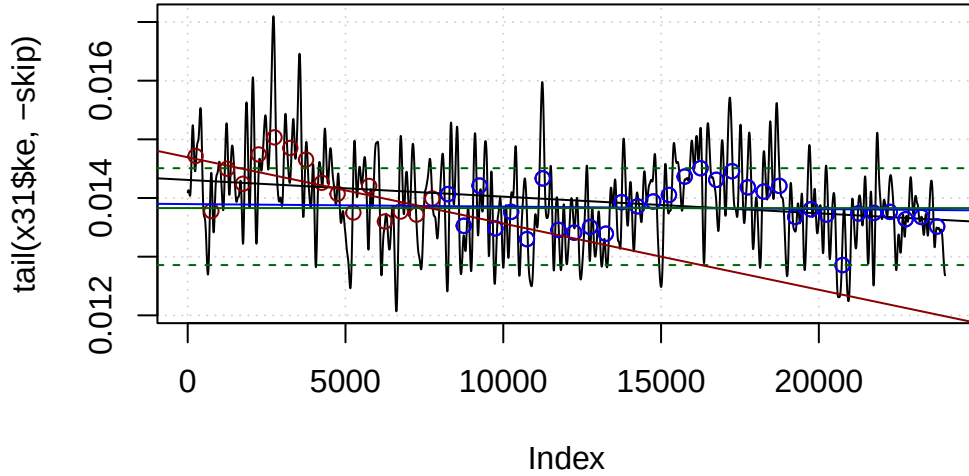


Figure 2: S2.0031 (Figure 1) kinetic energy (less the first 2,000 points) with fits to the resampled data, the black line is the fit to all of the data (red and blue circles), and the last two-thirds of truncated run S2-0031 (blue circles and line). Green lines indicate the mean (solid), and min & max (dashed) of the blue points. The red line is the regression line for the red points.

One other concern here is that `procMD2DF` did not extend the fit. It should have added 8 points to the fit, (the red points to the left of the blue points below the top green dashed line). These are just below the threshold we set for acceptance though ($F_{pvalue} < 0.5$).

```
lmMD2( tail( rx31, 40))
```

nr	Estimate	Std.Error	r.squared	Fval
4.000000e+01	-7.417483e-09	9.855196e-09	1.468834e-02	5.664774e-01
Fpvalue	df.dendf			
4.562996e-01	3.800000e+01			

Error in MD2DF

In the process of understanding the above, an error was noted in `procMD2DF` procedure to extend the data sets. Briefly, the error did searched from the beginning of the first third data

instead of searching backwards from the end. The result is that some of the averages may be on shorter data than need be. The data is otherwise valid, but this error obviates the issue of how to replace only those runs which were extended. The entire data frame needs to be re-created.

```
getwd()
```

```
[1] "C:/Users/phils/Documents/DSP/MD/MD2"
```

```
files = list.files("H:/Data/DSP/MD/Data/1980", "S2.*.scalars.csv")
length(files)
```

```
[1] 366
```

```
MD2DF <- createdf( "H:/Data/DSP/MD/Data/1980", files, clusters=num_cores,
                  rsblk=500)
```

```
sdf: Total Size: 11000, Step: 2048, Number of FFTs: 4, window Power: 4096
```

```
#save( MD2DF, file="MD2DF.4.S2.Rdata")
```

These results reflect the extended data..

```
cat( "Observations: ", nrow(MD2DF),
     ", Valid: ", sum( MD2DF[, 'Fpvalue'] >= 0.5))
```

```
Observations: 100 , Valid: 94
```

```
MD2DF <- MD2DF[ order( MD2DF$TE.mean.rs),]
op= par( mfrow = c(4, 2), mar = c(3, 3, 0.5, 0.5), tcl = -0.3,
        mgp = c(1.7, 0.4, 0) )
plotScalars( subset( MD2DF, TE.mean.rs< 0.2))
par(op)
```

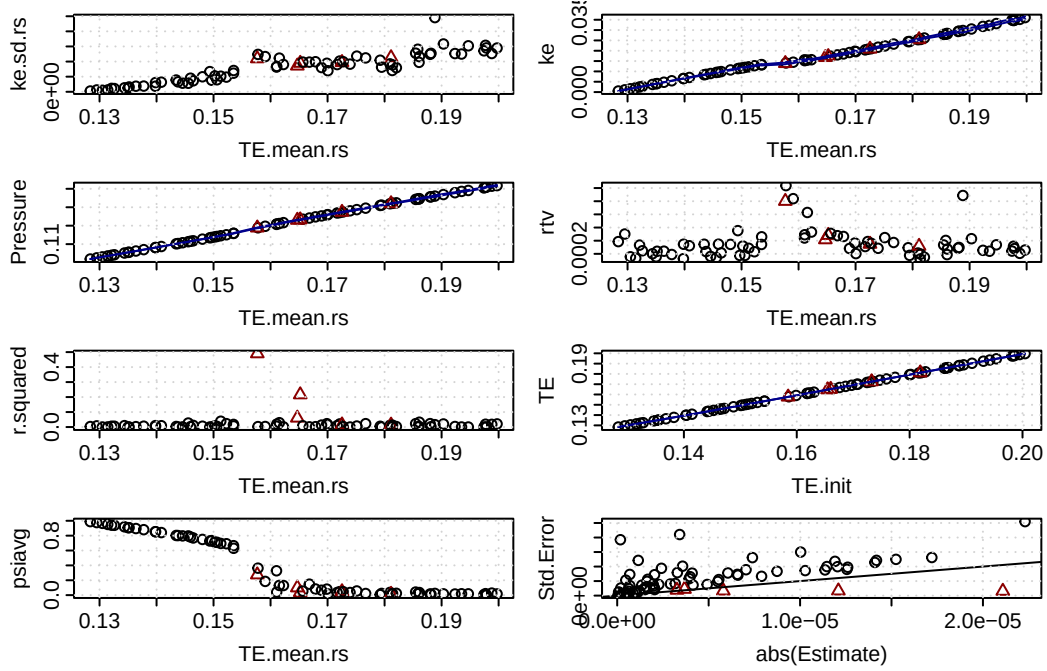


Figure 3: Eight key parameters, the first six (counting top-to-bottom, left-to-right) plotted against the mean total energy per particle. They are 1) the standard deviation of the kinetic energy, 2) the pressure, 3) the correlation coefficient, 4) the average of the modulus of the Nelson-Halperin order parameter, 5) the kinetic energy, and 6) the relative variance of the kinetic energy (i.e., the standard deviation divided by the mean). Then the total energy vs. the initialization total, and last the standard error of the slope estimate vs. the absolute value of the slope (with a line showing where they are equal). Black circles indicate data declared valid and red triangles are data whose regression had too much significance (F-test p-value < 0.5).

State of Main Scalars

Below are plots of the main scalars. The kinetic energy can be seen to stop increasing

```
plotconfMD2( df = MD2DF, y='ke', linethresh = 200, vpch=1,
             xlab = "Energy per Particle", ylab = "Kinetic Energy",
             main="Kinetic Energy vs. Total Energy")
```

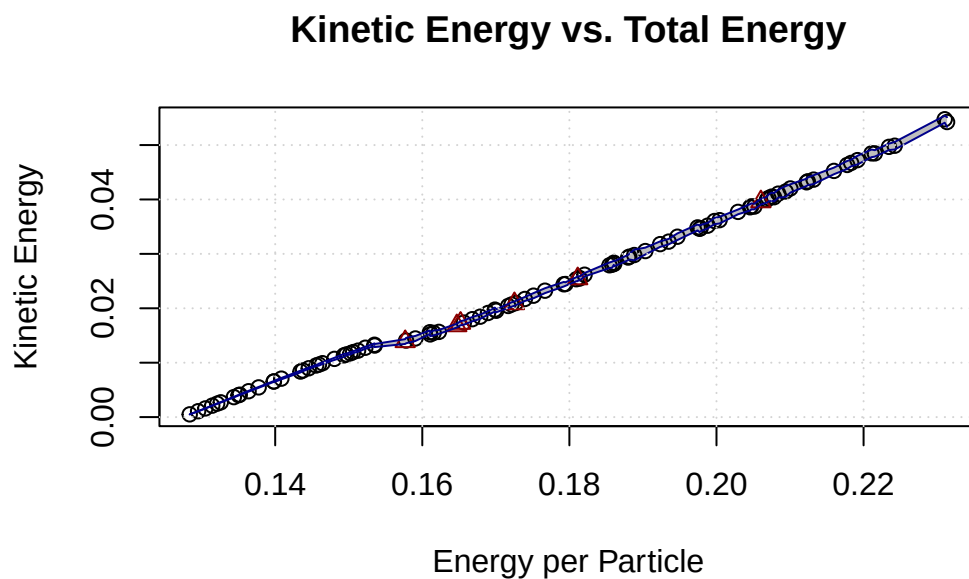


Figure 4: The per-particle kinetic energy versus total energy (energy in scaled units). Red triangles denote points not considered valid.

```
plotconfMD2( df = MD2DF[MD2DF$TE.mean.rs < 0.17,], y='ke', vpch=1,
             xlab = "Energy per Particle", ylab = "Kinetic Energy",
             main="Kinetic Energy vs. Total Energy")
```

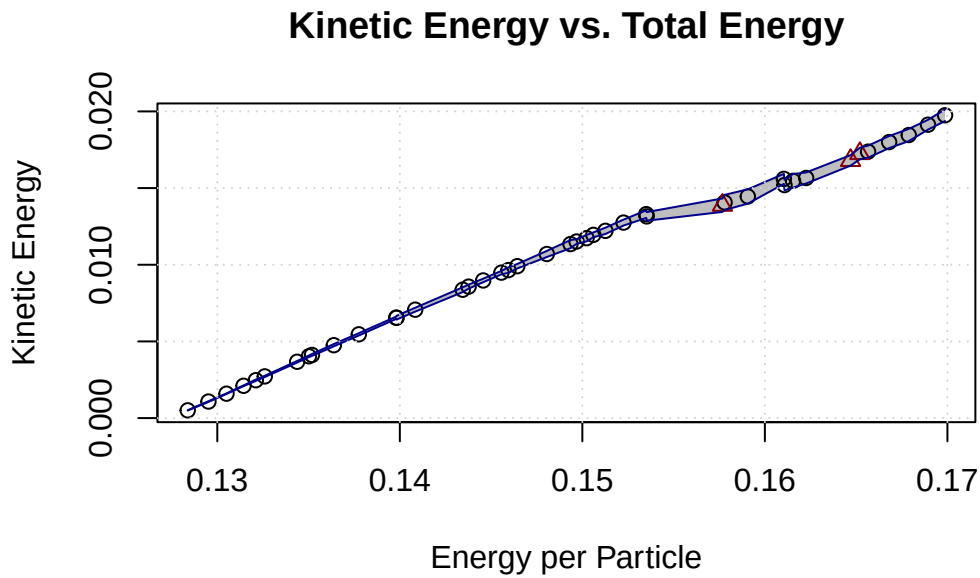



Figure 5: Figure 4 zoomed into the transition region.

The break is at point 29 to 30:

```
MD2DF$TE.mean.rs[29:30]
```

```
[1] 0.1535243 0.1576751
```

Point 30 here is not considered valid.

```
valid <- MD2DF$Fpvalue >= 0.5
pch <- rep_len( 1, length.out = nrow( MD2DF))
pch[ !valid] <- 2
col <- rep_len( 'black', length.out = nrow(MD2DF))
col[ !valid] <- 'darkred' # change all non-valid points to red.
x <- MD2DF[, 'TE.mean.rs']
Fpvalue <- MD2DF[, 'Fpvalue']
plotconf( y = MD2DF[, 'rtv'], x = x, Fpvalue = Fpvalue, linethresh = 200,
          xlab = "Energy per Particle",
          ylab = "Relative Variance", main="Relative KE variance vs. Total Energy")
```

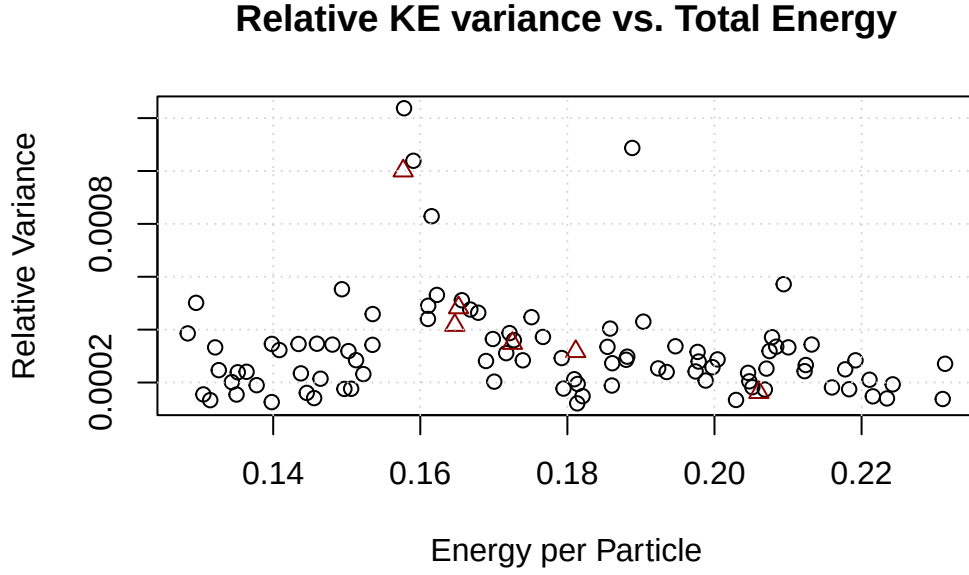


Figure 6: The per-particle kinetic energy versus total energy (energy in scaled units). Red triangles denote points not considered valid.

Certainly those points above 6×10^{-3} should be examined for equilibrium, perhaps all those above 4×10^{-3}

The Nelson-Halperin order parameter $\psi = \frac{1}{2(N-1)} \sum_{i \neq j} e^{i6\theta_{ij}}$, where θ_{ij} is the angle with the x -axis of the vector from particle i to j (the factor of 2 because in the above, each particle is counted twice).

```
plotconfMD2( df= MD2DF, y = 'psiabs', linethresh = 200,
             main="NHop vs. TE", xlab = "TE")
```

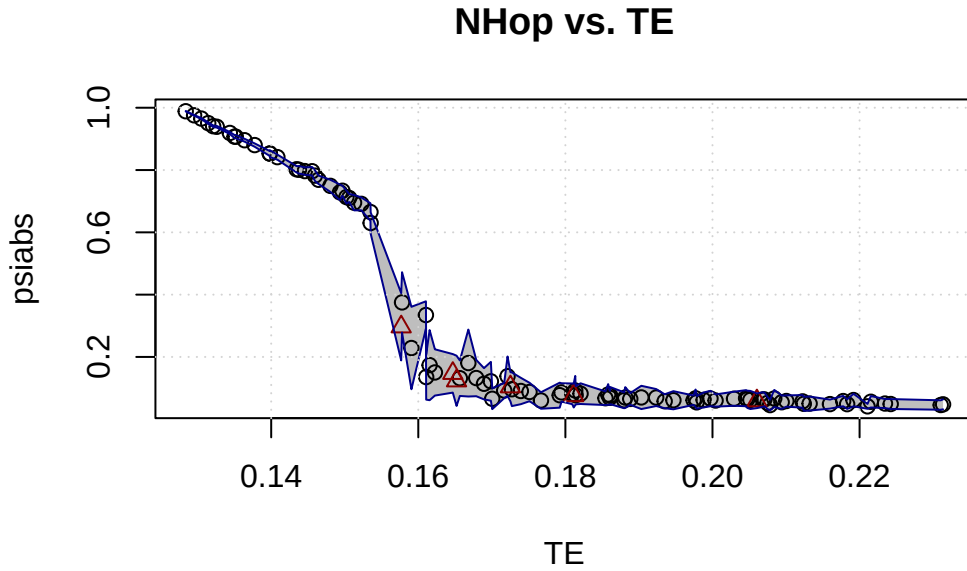


Figure 7: The Nelson-Halperin order parameter versus TE (energy in scaled units). Red triangles denote points not considered valid.

The system is started with the particles in minimum potential energy positions but with random kinetic energy. The kinetic energy quickly moves into average potential energy, and the plot below shows how this varies with total energy in the system. The plot is the ratio of the gain in potential energy (over the rest position) divided by the average kinetic energy. The ratio remains near one, but the peak in the transition region is perhaps as sharp as the Nelson-Halperin order parameter.

```
a <- sqrt( pi / sin(pi/3))
b <- a * sqrt(3)
pe.init <- (6/2) * ((a^-5) + (b^-5))
MD2DF$pe.gain <- MD2DF$pe.mean.rs - pe.init
#plot( pe.gain/ke.mean ~ ke.mean, data=MD2DF.2)
with( MD2DF,{
plotconf( y =pe.gain/ke.mean.rs, x = TE.mean.rs, Fpvalue = Fpvalue,
          linethresh = 200, xlab = "TE", ylab = "Fractional PE Gain",
          main="PE Gain vs. TE")
})
```

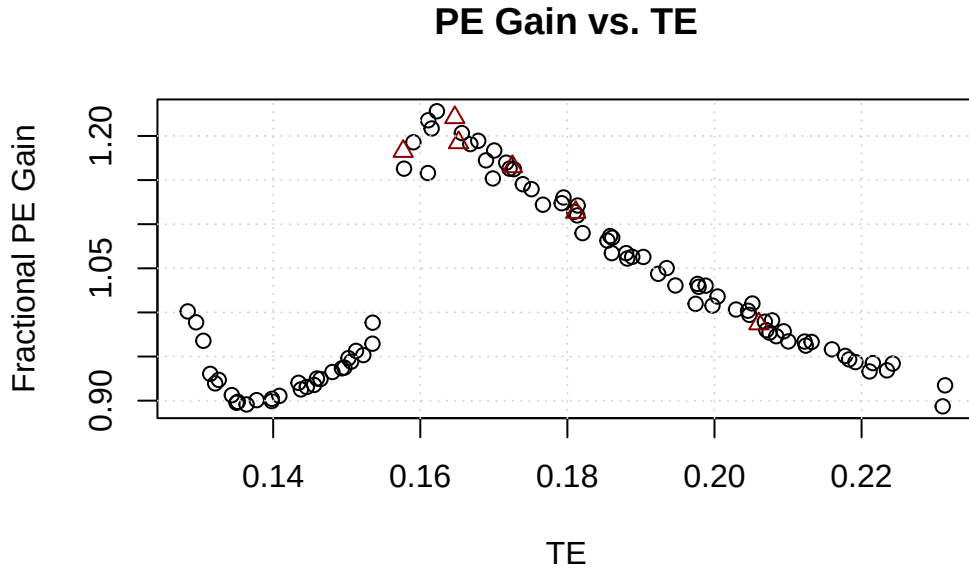


Figure 8: The ratio of gain in potential energy to the kinetic energy plotted vs. TE (energy in scaled units). Red triangles denote points not considered valid.

Combining the relative KE variance with the F-test, we can generate a list of suspect runs.

```
(rerun <- with( MD2DF, {MD2DF[ ((Fpvalue < 0.5) | (rtv > 0.0006)),
                                c('series', 'energy', 'TE.mean.rs', 'psiavg',
                                  'Fpvalue')]}))
```

	series	energy	TE.mean.rs	psiavg	Fpvalue
30	S2	0030	0.1576751	0.27446702	1.536416e-08
31	S2	0031	0.1577925	0.36276191	7.650440e-01
36	S2	0036	0.1590651	0.18444820	7.156608e-01
33	S2	0033	0.1615419	0.13026227	5.616665e-01
38	S2	0038	0.1646941	0.10011297	8.426301e-02
37	S2	0037	0.1652028	0.03653804	6.693158e-04
51	S2	0051	0.1725277	0.04828730	3.971003e-01
59	S2	0059	0.1811328	0.01371005	4.129641e-01
70	S2	0070	0.1888237	0.02297947	6.693438e-01
74	S2	0074	0.2060394	0.01244467	3.534746e-01

Character list of 10 runs to add steps to.

```
rerun$energy
```

```
[1] "0030" "0031" "0036" "0033" "0038" "0037" "0051" "0059" "0070" "0074"
```

Run 36 is just below 0.16ϵ energy per particle, so almost fluid.

```
x36 = combineruns( "H:/Data/DSP/MD/Data/1980", "S2\\.0036\\.")  
plotMD2( tail( x36, -2000), dt=0.01)
```

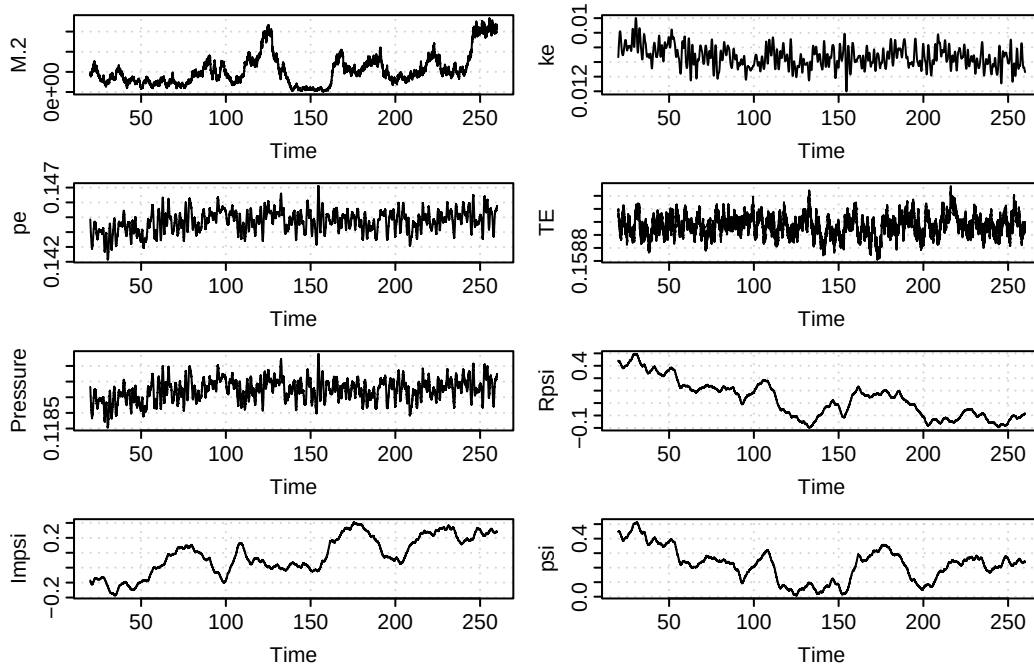


Figure 9: Plots of the basic data in run S2.0036 after skipping the first 2,000 points.

The Transition Region

We have enough data to identify the transition region. In addition to the gaps in the charts above, we can do a pair of fits on the pressure and temperature plots against the total energy. It is unclear exactly where the transition region starts, as we were unlucky with the total energies that were randomly generated.

```
plotconfMD2( df = MD2DF[MD2DF$TE.mean.rs %within% c(0.14,0.17),], y='ke', vpch=1,
  xlab = "Energy per Particle", ylab = "Kinetic Energy",
  main="Kinetic Energy vs. Total Energy")
```

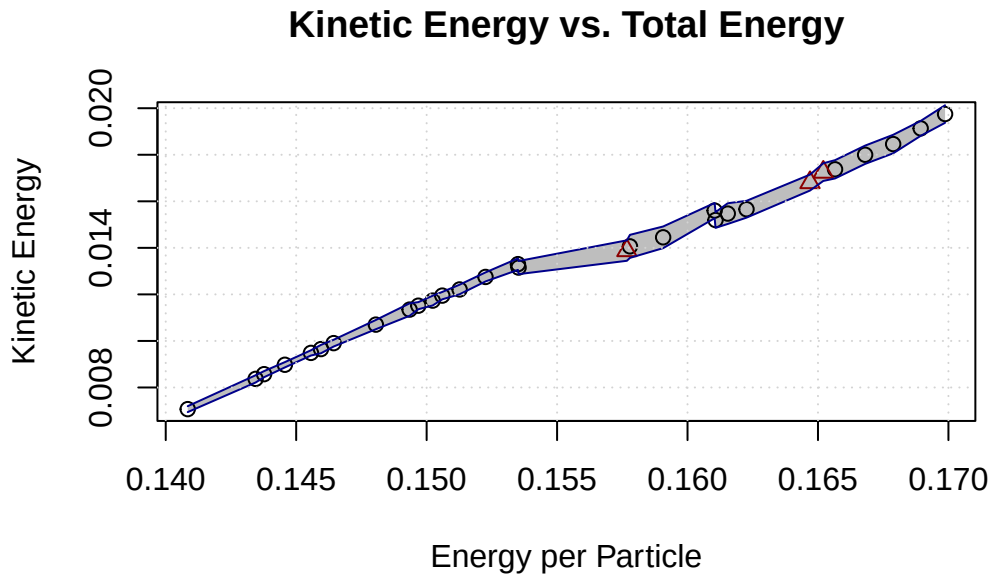


Figure 10: Figure 4 zoomed into the transition region.

Based on the above, the runs below 0.155 ϵ belong to the frozen (solid phase) systems, and the valid runs above 0.165 ϵ belong to the fluid phase.

```
vs <- MD2DF[, 'TE.mean.rs'] < 0.155
solidlm <- lm( ke.mean.rs ~ TE.mean.rs, data=MD2DF[ vs,])
(solidsum <- summary( solidlm))
```

Call:

```
lm(formula = ke.mean.rs ~ TE.mean.rs, data = MD2DF[vs, ])
```

Residuals:

	Min	1Q	Median	3Q	Max
	-2.947e-04	-4.976e-05	2.855e-05	6.918e-05	1.045e-04

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	-0.0649154	0.0003219	-201.6	<2e-16 ***
TE.mean.rs	0.5104236	0.0022671	225.1	<2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 9.594e-05 on 27 degrees of freedom

Multiple R-squared: 0.9995, Adjusted R-squared: 0.9994

F-statistic: 5.069e+04 on 1 and 27 DF, p-value: < 2.2e-16

```
vf <- with(MD2DF, TE.mean.rs > 0.165 & Fpvalue > 0.5)
fluidlm <- lm( ke.mean.rs ~ TE.mean.rs, data=MD2DF[vf,])
(fluidsum <- summary( fluidlm))
```

Call:

```
lm(formula = ke.mean.rs ~ TE.mean.rs, data = MD2DF[vf, ])
```

Residuals:

	Min	1Q	Median	3Q	Max
	-4.579e-04	-1.735e-04	-5.310e-06	1.344e-04	1.003e-03

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	-0.0764907	0.0003739	-204.6	<2e-16 ***
TE.mean.rs	0.5636935	0.0019030	296.2	<2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.0002631 on 57 degrees of freedom

Multiple R-squared: 0.9994, Adjusted R-squared: 0.9993

F-statistic: 8.774e+04 on 1 and 57 DF, p-value: < 2.2e-16

```
plot( ke.mean.rs ~ TE.mean.rs, data=MD2DF, pch=1, type='n')
points( ke.mean.rs ~ TE.mean.rs, data=MD2DF[ vs,], pch=1)
points( ke.mean.rs ~ TE.mean.rs, data=MD2DF[ vf,], pch=1, col='blue')
abline( coef=coef( solidlm))
abline( coef=coef( fluidlm), col='blue')
```

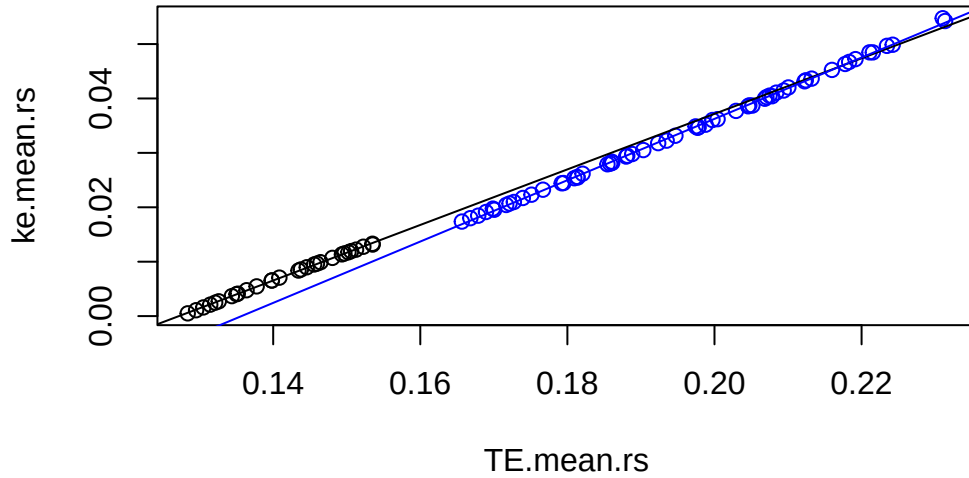


Figure 11: Linear fits to solid and fluid phases.

Note the specific heat appears to change a bit in the two phases: the solid is about 0.51, and the fluid is a bit higher at 0.56. The zero point energy is given by:

```
solidc <- coef( solidlm)
-unname(solidc['(Intercept)'] / solidc['TE.mean.rs'])
```

```
[1] 0.1271795
```

The zero point energy is compared to the exact value (to six digits) of 0.127373.

The heat of fusion can be given by the horizontal distance between the two lines. If the solid line is $y = a_s + b_s x$ and the fluid is similarly $y = a_f + b_f x$, then the x coordinate in the fluid phase corresponding to the an x_0 in the solid phase, is $\frac{a_s - a_f + b_s x_0}{b_f} - x_0$.

```
(t1 <- tail( sort(MD2DF[vs,"TE.mean.rs"]), 1))
```

```
[1] 0.1535243
```



```
(t2 <- head( sort( MD2DF[vf,"TE.mean.rs"]), 1))
```

```
[1] 0.1656538
```

```
fluidc <- coef( fluidlm)
af <- fluidc['(Intercept)']
bf <- fluidc['TE.mean.rs']
as <- solidc['(Intercept)']
bs <- solidc['TE.mean.rs']
f <- function( as, bs, af, bf, x0) (as - af + bs*x0)/bf - x0
tx <- seq( from=t1, to=t2, length.out=20)
plot( x=tx, y=f(as, bs, af, bf, tx), xlab='TE/part.', ylab='Heat of Fusion')
```

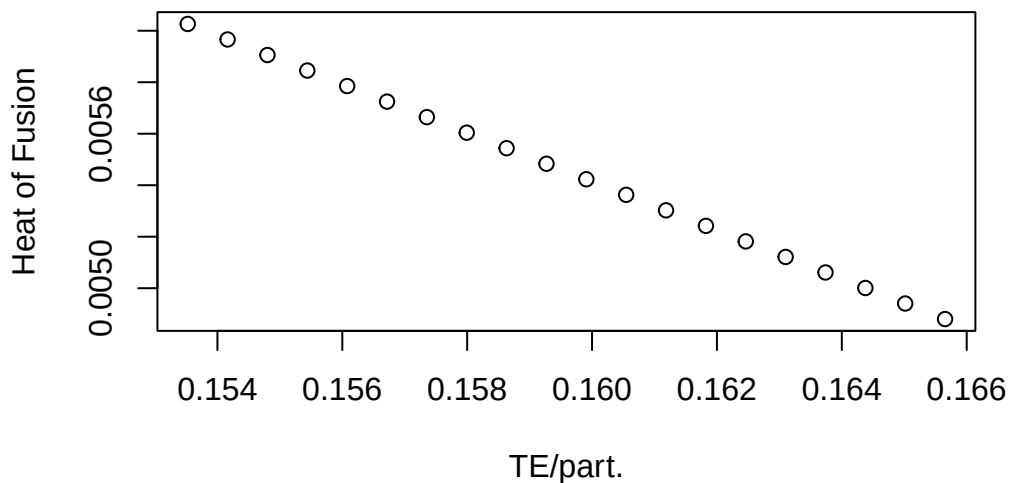


Figure 12: Range of heat of fusion.

The next series of runs needs to extend the runs elicited above, plus fill in the energies that are missing. The `InitMDRecord!` function was updated to zero in on the energies in a better way. Previously, the energy was set as a parameter to the random number generation, but the random generation could result in many different energies. The old code was as follows:

```
ss.v .= sqrt( Temp) .* randn( eltype( ss.v), ss.N)
```

The new code is as follows:

```
ss.v .= randn( eltype( ss.v), ss.N)
ke = 0.5 * sum( dot.(ss.v, ss.v)) / ss.N
ss.v .= sqrt( Temp / ke) .* ss.v
```

As can be seen, the random numbers are generated first, and then they are scaled to meet the temperature. It is still not exact, as linear and angular momentum are corrected after the kinetic energy generation, and this does change it a bit, but it ought to be a lot closer now.